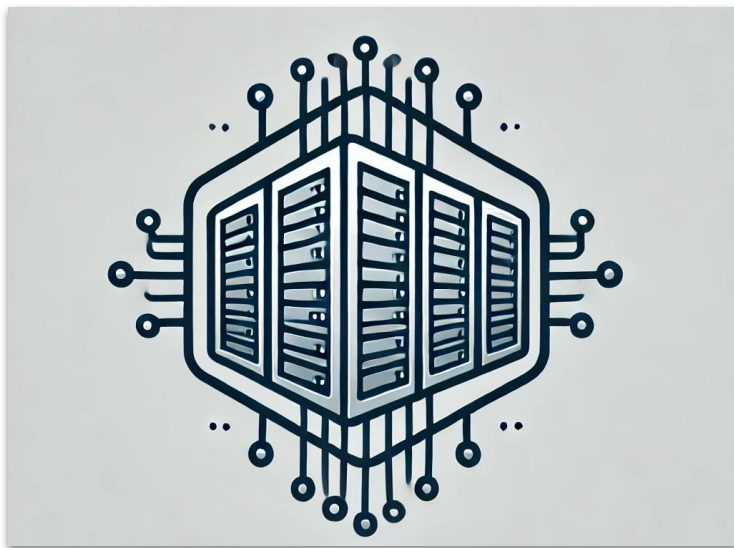


CSC 746: High Performance Computing

Optimizing use of cache

15 Sep 2026



CP1 Debrief, Ctd.

MFLOPS != “Throughput”

Fabrication of data is scientific/academic misconduct

Using words/terms you may not understand: AI tells you something that “sounds good” so you repeat it w/o understanding it

→ **Update to CP2**: you are restricted to the use of concepts we’ve discussed in lecture so far.

Hint: develop a writing “playbook”, “recipes”

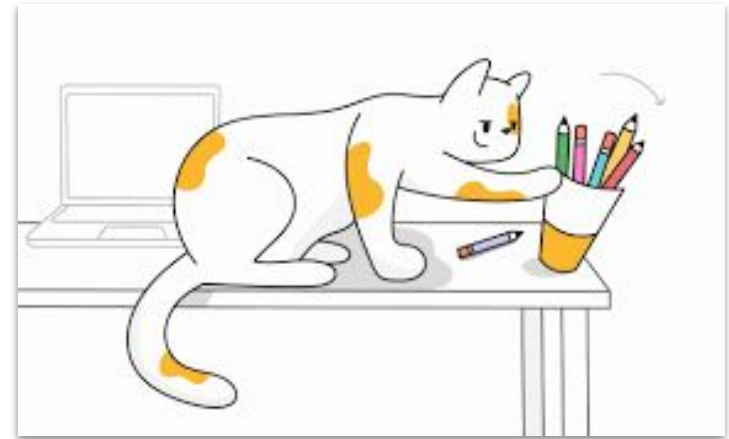
Today

Last time:

- Review of CP#2 block+copy methodology
- Cache-friendly memory layout (Z-order) case study

Today:

- Review previous work on methods for optimizations to make better use of cache



Upcoming dates/deadlines:

CP#2 due: 9/22/2025

CP#2 presentations: 9/23/2025

Volunteer to give a CP#2
presentation: 9/18/2025

Previous works studying cache optimization

The cache performance and optimizations of blocked algorithms. Lam, Rothberg, and Wolf. 4/1991, <https://dl.acm.org/doi/10.1145/106974.106981>

Locality Optimizations for Multi-level caches, Rivera and Tang, 1/1999, <https://dl.acm.org/doi/pdf/10.1145/331532.331534>

Cache-oblivious algorithms. Frigo, Leiserson, Prokop, and Ramchandran. 10/1999. <https://dl.acm.org/doi/10.5555/795665.796479>

What every programmer should know about memory, 11/2007, <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>

The Cache Performance and Optimizations of Blocked Algorithms (1991)

<https://dl.acm.org/doi/10.1145/106974.106981>

Key problems:

- FLOP rates increasing much faster than memory access rates
- Why do naïve loop nests (e.g., dense linear algebra) run far below peak even when compute is ample?
- How do blocking/tiling, loop interchange, and strip-mining change cache behavior (capacity vs. conflict misses)?
- How should one choose block sizes to exploit a given cache level?

Blocking – Compute on Subsets of Larger Arrays

Consider the example of matrix multiplication for matrices of size $N \times N$:

```
for i := 1 to N do
  for k := 1 to N do
    r = X[i,k]; /* register allocated */
    for j := 1 to N do
      Z[i,j] += r*Y[k,j];
```

Blocking applied at multiple levels in the memory hierarchy:

- Register blocking (above)
- Buffer blocking (right)

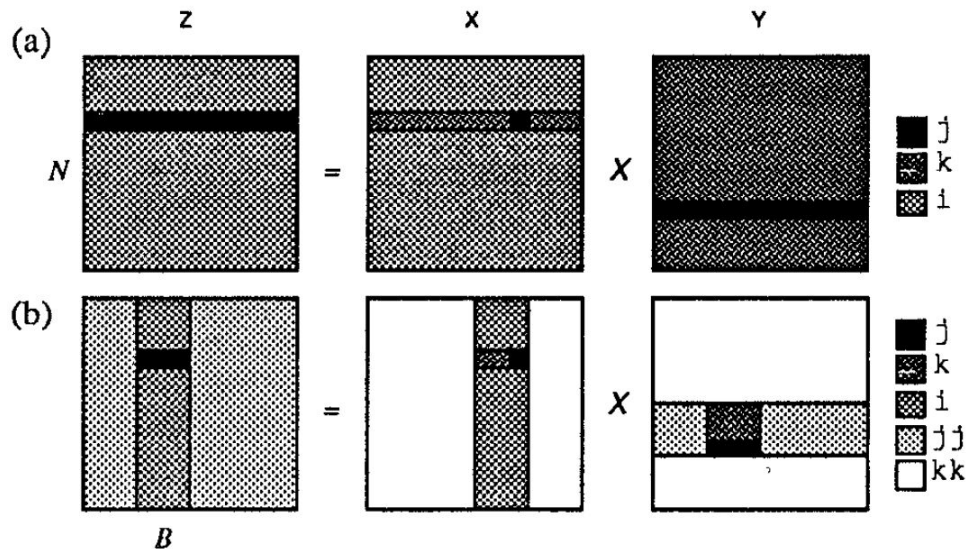


Figure 1: Data access pattern in (a) unblocked and (b) blocked matrix multiplication.

Blocking factor parameter: B

MM using register “blocking”

Blocking parameter: B

Question: is this a copy optimization?

```
for  $kk := 1$  to  $N$  by  $B$  do  
  for  $jj := 1$  to  $N$  by  $B$  do  
    for  $i := 1$  to  $N$  do  
      for  $k := kk$  to  $\min(kk+B-1, N)$  do  
         $r = X[i,k]$ ; /* register allocated */  
        for  $j := jj$  to  $\min(jj+B-1, N)$  do  
           $Z[i,j] += r * Y[k,j]$ ;
```



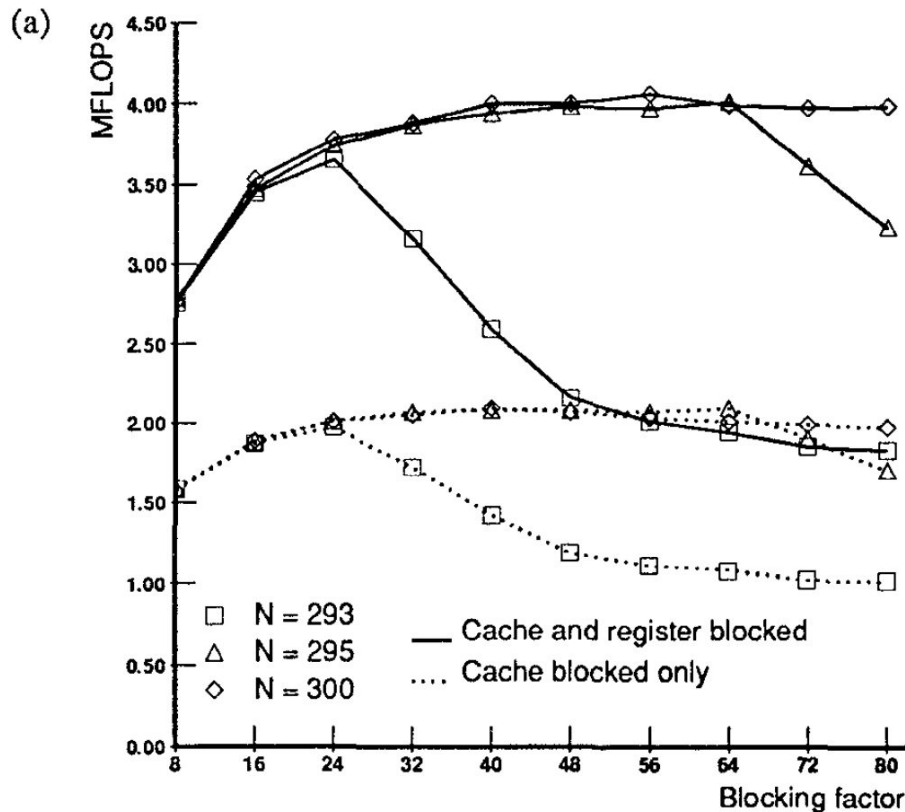
Blocking: 3-4x speedup

Platform:

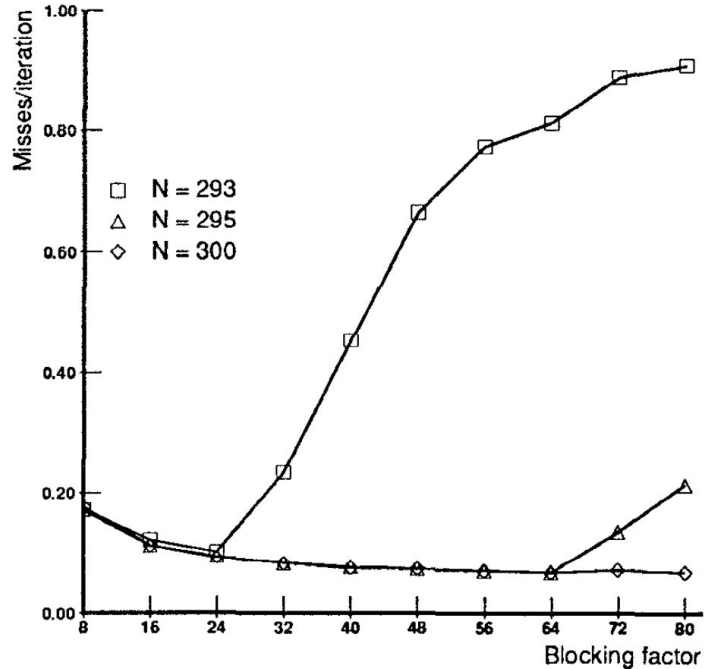
- DECStation 3100
- MIPS R2000 processor running at 16.67 MHz (thanks Gemini)
- 8K direct-mapped cache: holds all of an 88x88 block

Interplay between matrix size and performance:

- 300x300 – best perf, well behaved
- 295x295 – 50% of perf, well behaved
- 293x293 – poorly behaved (“interference misses”)

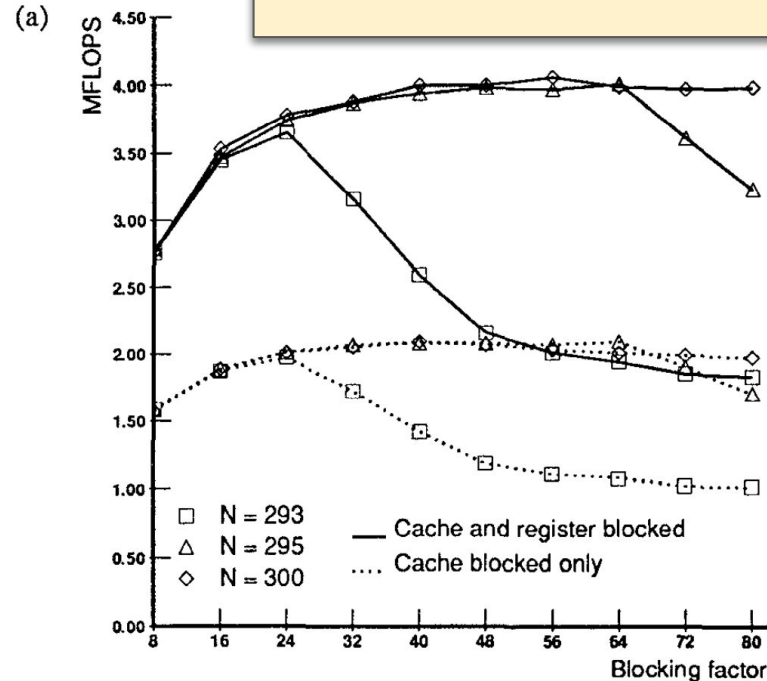


MFLOPS and Cache Miss rate



Interplay between matrix size and performance:

- 300x300 – best perf, well behaved
- 295x295 – 50% of perf, well behaved
- 293x293 – poorly behaved (interference misses)



Types of Cache Misses

Intrinsic misses: the “usual” cache miss, inversely proportional to size B

Interference misses: (aka conflict miss) occurs when a requested data item is evicted from the cache to make space for a new item, even though the cache is not yet full.

- Self-interference misses: This happens when different data from the same array or variable map to the same cache location and evict each other.
- Cross-interference misses: when data from different arrays or variables map to the same cache location and evict each other.

Types of Cache Misses

Intrinsic misses: the “usual” cache miss, inverse

Interference misses: (aka conflict miss) occurs when data is evicted from the cache to make space for a new item, even though the cache is not yet full.

- Self-interference misses: This happens when two different array or variable map to the same cache location.
- Cross-interference misses: when data from two different arrays map to the same cache location and evict each other.

The focus of the paper goes on to study the behavior of these different types of misses using a simulated model of cache where they can vary cache parameters.

They are basing their work on the direct-mapped cache model

Eg, “we have developed an algorithm to find the largest blocking factor B that avoids self interference for a given matrix size”

Analyzing Misses

Prediction of different types of misses on a machine with $C=1K$ words of memory

Intrinsic: misses decrease with increasing B up to limit of $\sqrt{C}$

Self-interference: (conflicts for one array) there is a critical blocking factor B_0 where $B > B_0$ leads to increased self interference

Cross interference: (conflicts between arrays) tapers off after self-interference kicks in

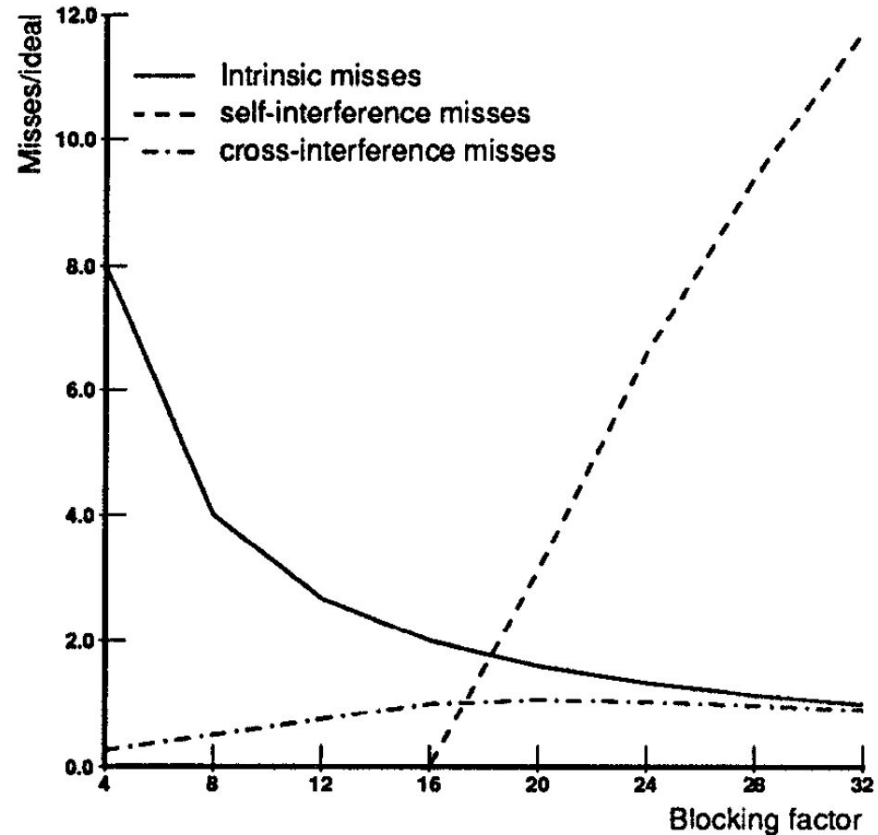


Figure 4: Breakdown of misses, 1K word cache, $N = 295$.



Actual vs Predicted

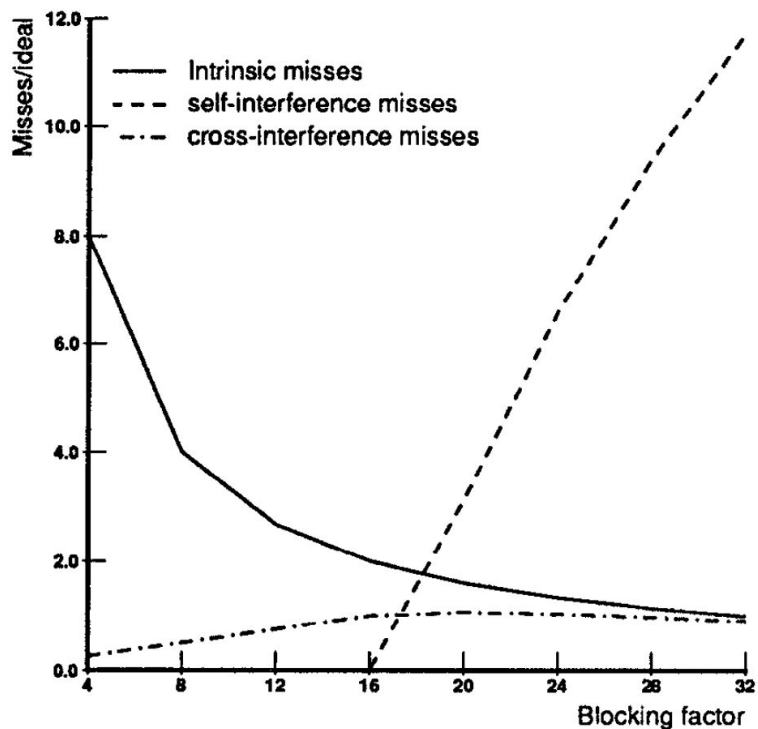


Figure 4: Breakdown of misses, 1K word cache, $N = 295$.

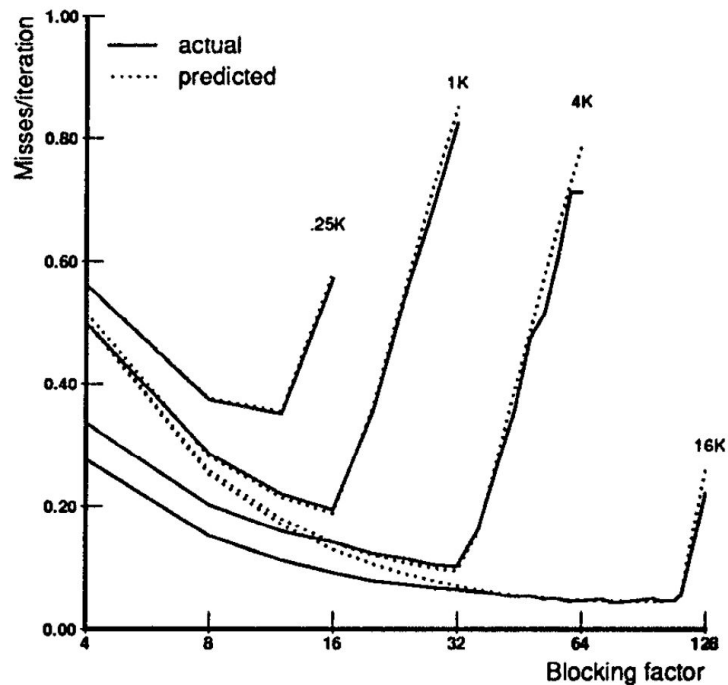


Figure 5: Actual versus predicted cache misses across different cache sizes (in words), $N = 295$.

Choosing the Blocking Factor

Predicted miss rate as a function of block size B for a 1K cache

For each B , vary the size of N to produce a distribution of cache miss rates

$B=8$ slightly higher than $B=12$ but much smaller deviation

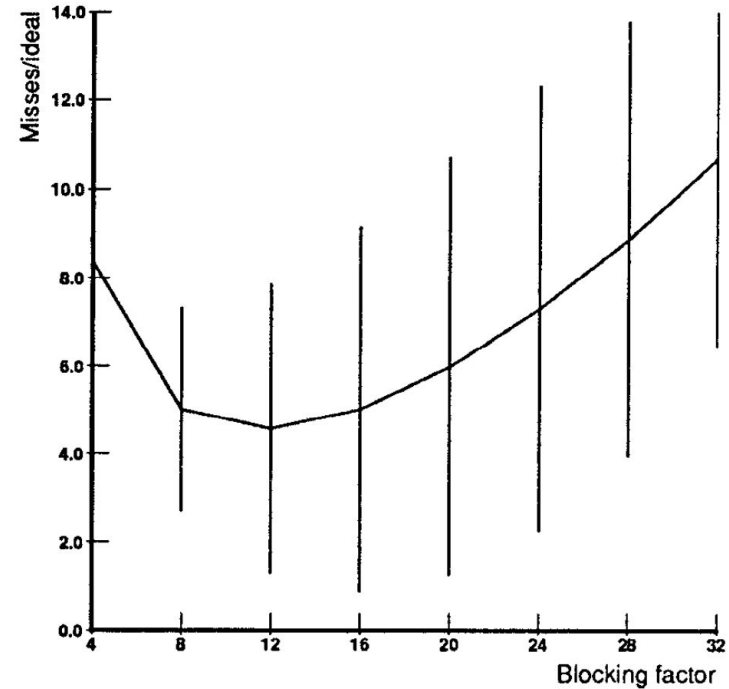


Figure 6: Average misses in 1K word cache. Vertical lines indicate the standard deviation.

Choosing the Blocking Factor

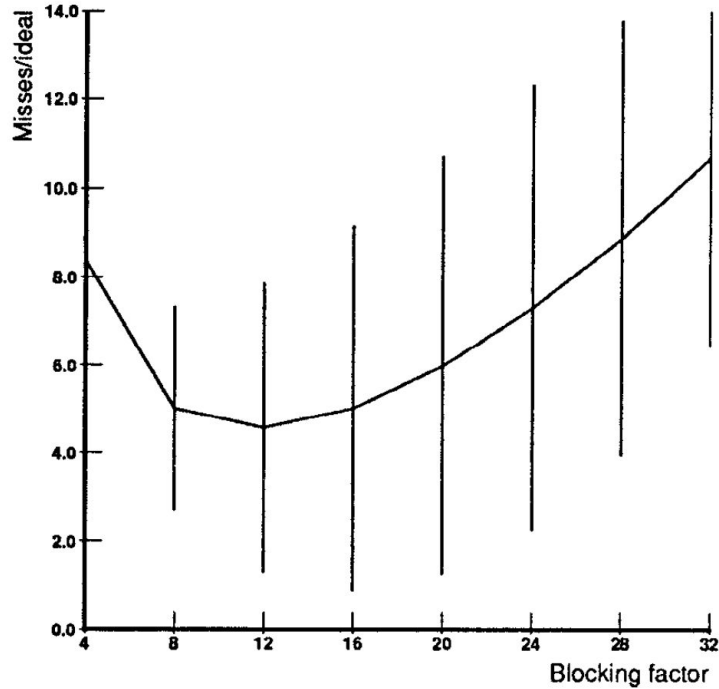


Figure 6: Average misses in 1K word cache. Vertical lines indicate the standard deviation.

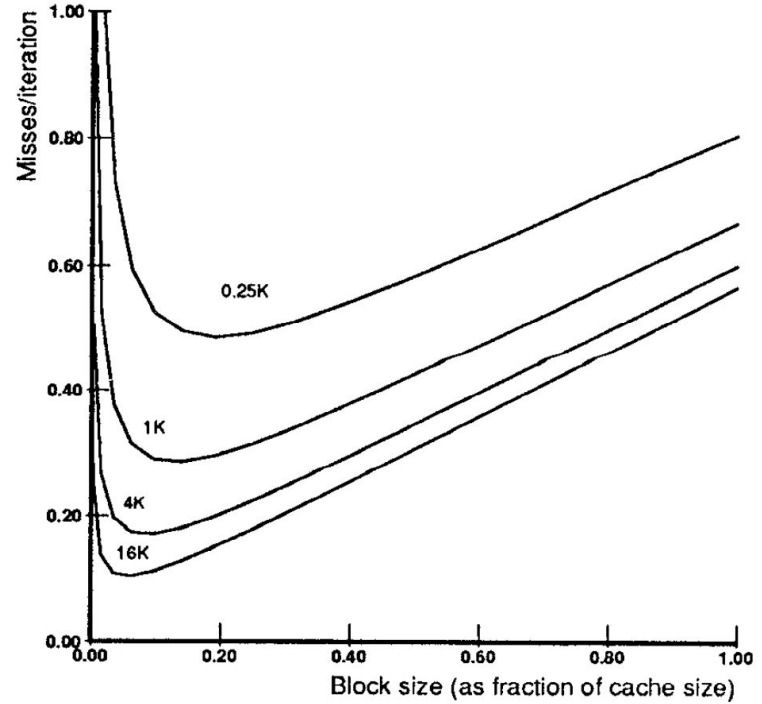


Figure 7: Miss rate (averaged over all matrix sizes) versus block size, across different cache sizes (in words).

Block vs. Block+Copy

MM using register “blocking”

Blocking parameter: B

This code operations directly to/from the matrices X , Y , and Z

There is no “copy” optimization here

How will adding the copy optimization change the performance?

```
for  $kk := 1$  to  $N$  by  $B$  do  
  for  $jj := 1$  to  $N$  by  $B$  do  
    for  $i := 1$  to  $N$  do  
      for  $k := kk$  to  $\min(kk+B-1, N)$  do  
         $r = X[i,k]$ ; /* register allocated */  
        for  $j := jj$  to  $\min(jj+B-1, N)$  do  
           $Z[i,j] += r * Y[k,j]$ ;
```

Block+Copy

Adding the copy optimization eliminates self-interference from the misses

Approach: copy non-contiguous data into a contiguous memory buffer

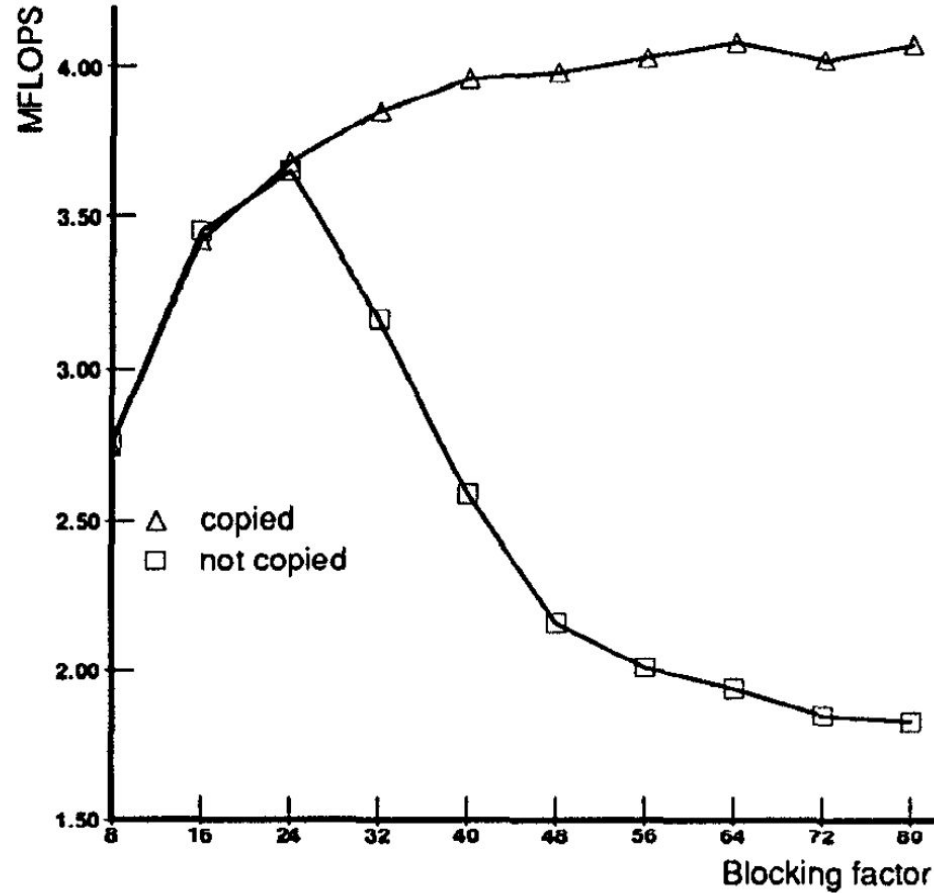


Figure 17: Performance of copying versus no copying, $N = 293$, DECstation 3100.



The Cache Performance and Optimizations of Blocked Algorithms (1991)

Key findings

- Blocking dramatically reduces capacity misses by keeping working sets resident at a chosen cache level; speedups of several $\times$ are routine for matrix kernels.
- A simple working-set model (tile of size $b \times b$) predicts good b values when sized to a fraction of the target cache.
- Loop interchange to make the inner loop unit-stride plus padding to avoid set conflicts materially improves performance.
- Adding the copy optimization further improves performance by minimizing self-interference effects

Previous works studying cache optimization

The cache performance and optimizations of blocked algorithms. Lam, Rothberg, and Wolf. 4/1991, <https://dl.acm.org/doi/10.1145/106974.106981>

Locality Optimizations for Multi-level caches, Rivera and Tang, 1/1999, <https://dl.acm.org/doi/pdf/10.1145/331532.331534>

Cache-oblivious algorithms. Frigo, Leiserson, Prokop, and Ramchandran. 10/1999. <https://dl.acm.org/doi/10.5555/795665.796479>

What every programmer should know about memory, 11/2007, <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>

Locality Optimizations for Multi-Level Caches (SC'99)

<https://dl.acm.org/doi/pdf/10.1145/331532.331534>

Key problems

- Which compiler/program transformations (tiling, fusion, padding, alignment, loop distribution) matter most when caches are multi-level?
- Should we target one level (e.g., L1) or attempt multi-level blocking?
- How do transformations trade off temporal vs. spatial locality across producer–consumer phases?

Loop Nest and Data Layout Transformations

<pre>// Original real A(N,M), B(N) do j = 1,N do i = 1,M B(j) = A(j,i)</pre>	<pre>// Loop Permutation real A(N,M), B(N) do i = 1,M do j = 1,N B(j) = A(j,i)</pre>	<pre>// Array Transpose real A(M,N), B(N) do j = 1,N do i = 1,M B(j) = A(i,j)</pre>
--	--	---

Figure 1 Loop Nest and Data Layout Transformations

Intervariable Padding

Unit stride references to A, B
provide spatial locality

On a DM cache systems, if
addresses of A and B differ by a
multiple of cache size, will have
cross interference

Idea: modify addresses of A and B
to eliminate/minimize cross
interference

```
real A(N,N), B(N,N), C(N,N)

do j = 2,N-1      // loop nest 1
  do i = 1,N
    = A(i,j) + A(i,j+1)
    = B(i,j) + B(i,j+1)
    = C(i,j) + C(i,j+1)

do j = 2,N-1      // loop nest 2
  do i = 1,N
    = B(i,j-1) + B(i,j) + B(i,j+1)
    = C(i,j)
```

Figure 2 Example program

Intervariable padding, ctd

Compiler code analysis:

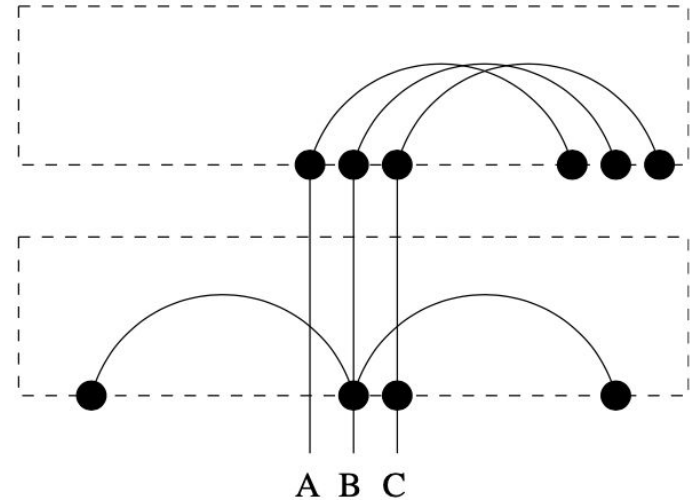
- Study array access patterns
- Adjust base addresses of arrays to find a pattern with minimum cross interference

Assume before picture: all $\text{addr} \% \text{cache_size} == 0$

→ the dots would all land on top of each other

Top: first loop nest, array addressed nudged into new positions so they don't collide

Bottom: second loop nest, no “group reuse” of B column j with j+1 due to presence of C in cache



```
real A(N,N), B(N,N), C(N,N)

do j = 2,N-1    // loop nest 1
  do i = 1,N
    = A(i,j) + A(i,j+1)
    = B(i,j) + B(i,j+1)
    = C(i,j) + C(i,j+1)

do j = 2,N-1    // loop nest 2
  do i = 1,N
    = B(i,j-1) + B(i,j) + B(i,j+1)
    = C(i,j)
```

Figure 2 Example program

Loop Fusion

Transformation where adjacent loops are fused into a single loop

Idea: improve locality by bringing together memory references, or enable other transformations

Eliminates redundant accesses:

→ eg, $B(i,j)$

May increase chance of severe conflicts

```
real A(N,N), B(N,N), C(N,N)

do j = 2,N-1      // loop nest 1
  do i = 1,N
    = A(i,j) + A(i,j+1)
    = B(i,j) + B(i,j+1)
    = C(i,j) + C(i,j+1)

do j = 2,N-1      // loop nest 2
  do i = 1,N
    = B(i,j-1) + B(i,j) + B(i,j+1)
    = C(i,j)
```

Figure 2 Example program

```
real A(N,N), B(N,N), C(N,N)
do j = 2,N-1
  do i = 1,N
    = A(i,j) + A(i,j+1)
    = B(i,j) + B(i,j+1)
    = C(i,j) + C(i,j+1)
    = B(i,j-1) + B(i,j) + B(i,j+1)
    = C(i,j)
```

Figure 6 Example program after fusion



Tiling (Blocking)

A transformation combining *strip mining* with loop permutation to form small tiles of loop iterations

Idea: increase locality

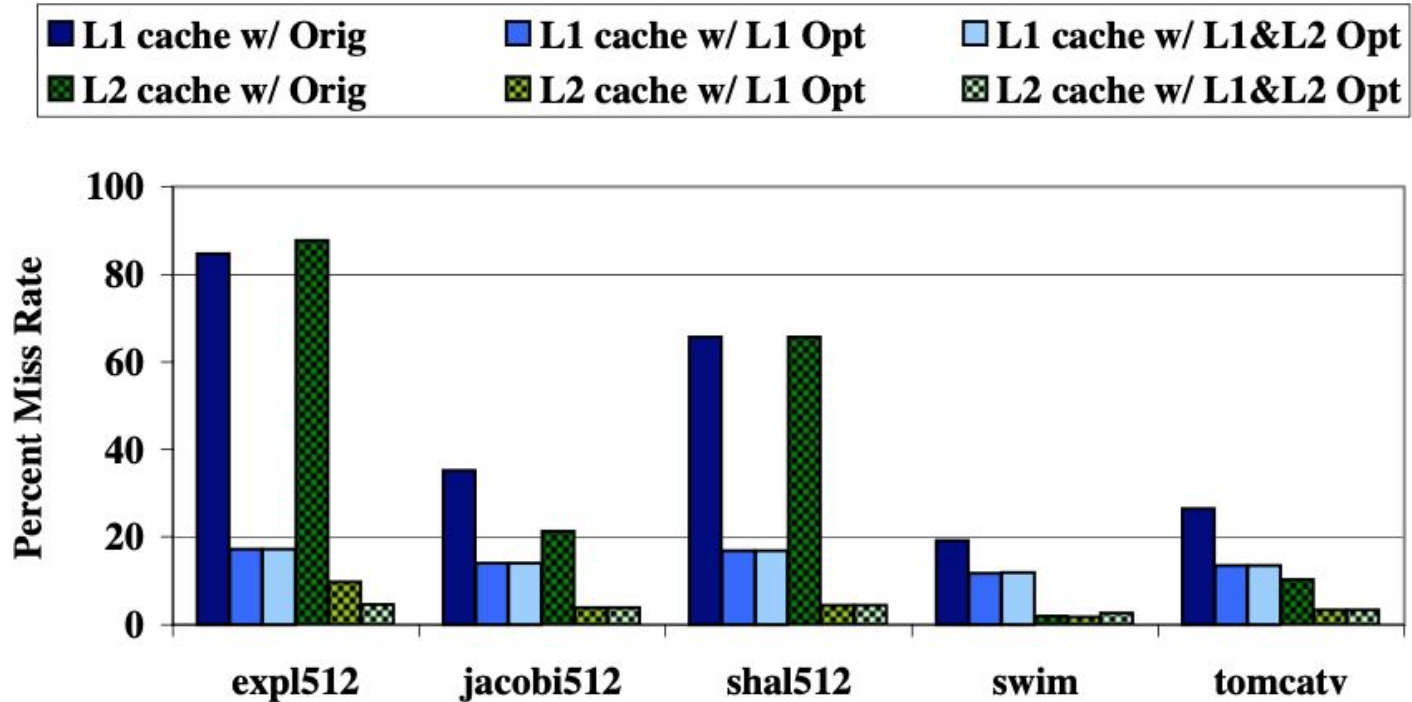
What's the “best tile size”?

- Tradeoff: smaller tiles increase overhead, larger tiles increase likelihood of cache overflow
- Question is more complex in multi-level cache configs
- They suggest: choose tile sizes targeting L1 cache size

```
do KK=1,N,W           // W = tile width
  do II=1,N,H          // H = tile height
    do J=1,N
      do K=KK,min(KK+W-1,N)
        do I=II,min(II+H-1,N)
          C(I,J) = C(I,J) + A(I,K)*B(K,J)
```

Figure 8 Tiled matrix multiplication

Some Results



More Results

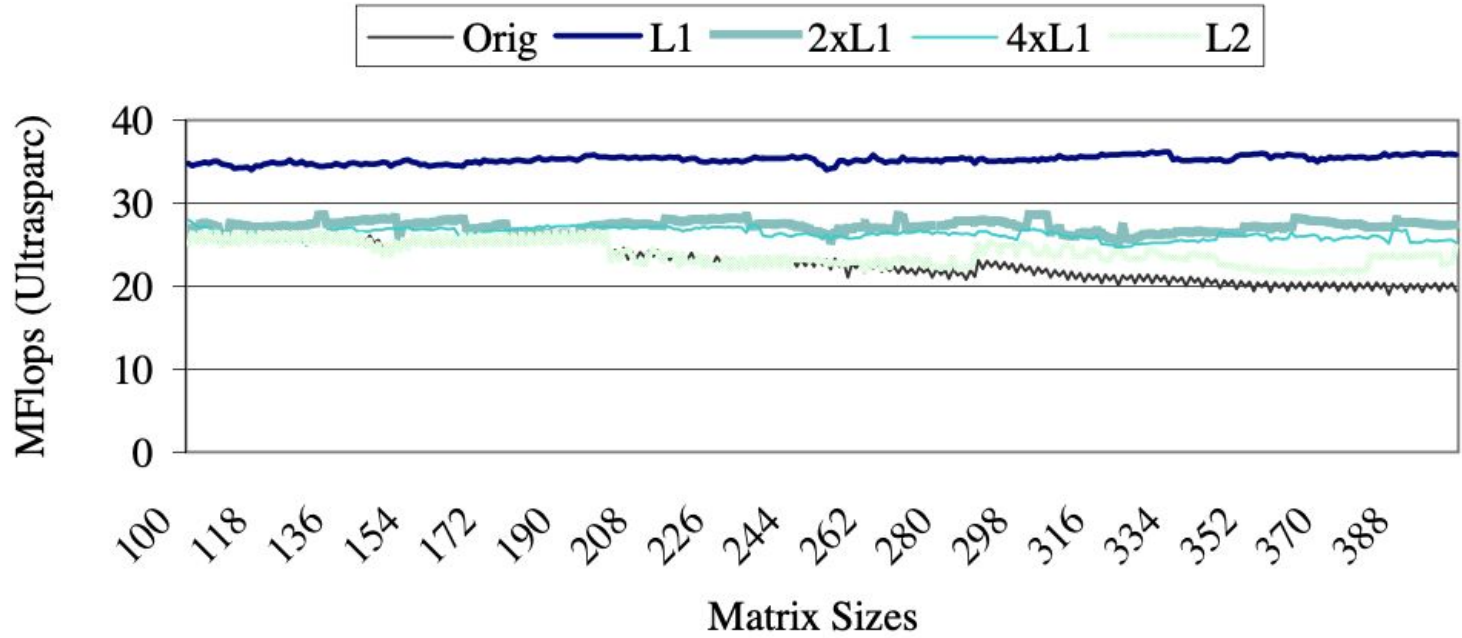


Figure 13 Performance for two tiling methods over varying problem size

Previous works studying cache optimization

The cache performance and optimizations of blocked algorithms. Lam, Rothberg, and Wolf. 4/1991, <https://dl.acm.org/doi/10.1145/106974.106981>

Locality Optimizations for Multi-level caches, Rivera and Tang, 1/1999, <https://dl.acm.org/doi/pdf/10.1145/331532.331534>

Cache-oblivious algorithms. Frigo, Leiserson, Prokop, and Ramchandran. 10/1999. <https://dl.acm.org/doi/10.5555/795665.796479>

What every programmer should know about memory, 11/2007, <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>

Cache-Oblivious Algorithms (1999)

<https://dl.acm.org/doi/10.5555/795665.796479>

Key problems

- Can we design algorithms that are asymptotically optimal for all cache levels without knowing cache sizes/line sizes?
- What are the miss bounds of divide-and-conquer (recursive) algorithms under the ideal-cache model and the tall-cache assumption?

Approach:

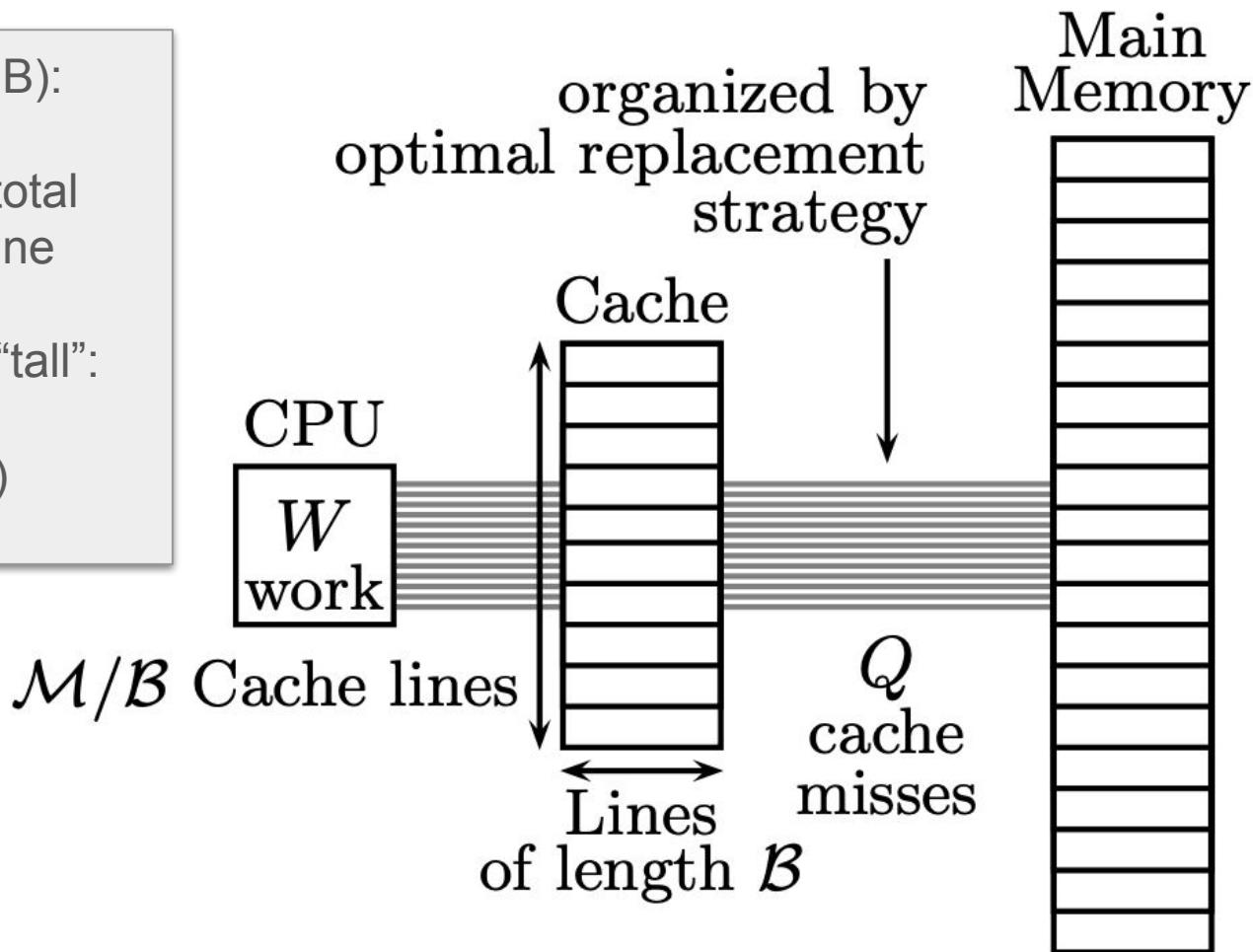
- Use divide-conquer + recursion as the basis for subdivision, hope to have the problem fit entirely in cache at some level of recursion

Cache(M,B):

M words total
B words/line

Cache is “tall”:

$$M = O(B^2)$$



Definitions

Cache-aware algorithm

- One that contains parameters that can be tuned to optimize performance for given cache size M and block size B
- Example: MMUL with blocking/tiling: choice of block size matters a lot

Cache-oblivious algorithm

- One that has no such tunable parameters

Cache Oblivious MMUL: Rect-Mult

Assume $A[M][N]$ and $B[N][P]$

Divide and conquer:

- Partition problem along largest of $[M, N, P]$
- Recurse and perform Rect-Mult on smaller matrices
- Stop recursion when conditions met and just do MMUL on partitioned matrices

Idea: Multiply $C = AB$, with $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times p}$. **Recursively halve the largest of $\{m, n, p\}$.** Once a subproblem fits in cache, its children do too.

If $m \geq \max\{n, p\}$:

$$A = \begin{pmatrix} A_1 \\ A_2 \end{pmatrix}, \quad C = AB = \begin{pmatrix} A_1 B \\ A_2 B \end{pmatrix}$$

Two independent multiplies; stack results vertically.

If $n \geq \max\{m, p\}$:

$$A = \begin{pmatrix} A_1 & A_2 \end{pmatrix}, \quad B = \begin{pmatrix} B_1 \\ B_2 \end{pmatrix}, \quad C = A_1 B_1 + A_2 B_2$$

Two multiplies **plus an add** into the same C .

If $p \geq \max\{m, n\}$:

$$B = \begin{pmatrix} B_1 & B_2 \end{pmatrix}, \quad C = AB = \begin{pmatrix} AB_1 & AB_2 \end{pmatrix}$$

Results

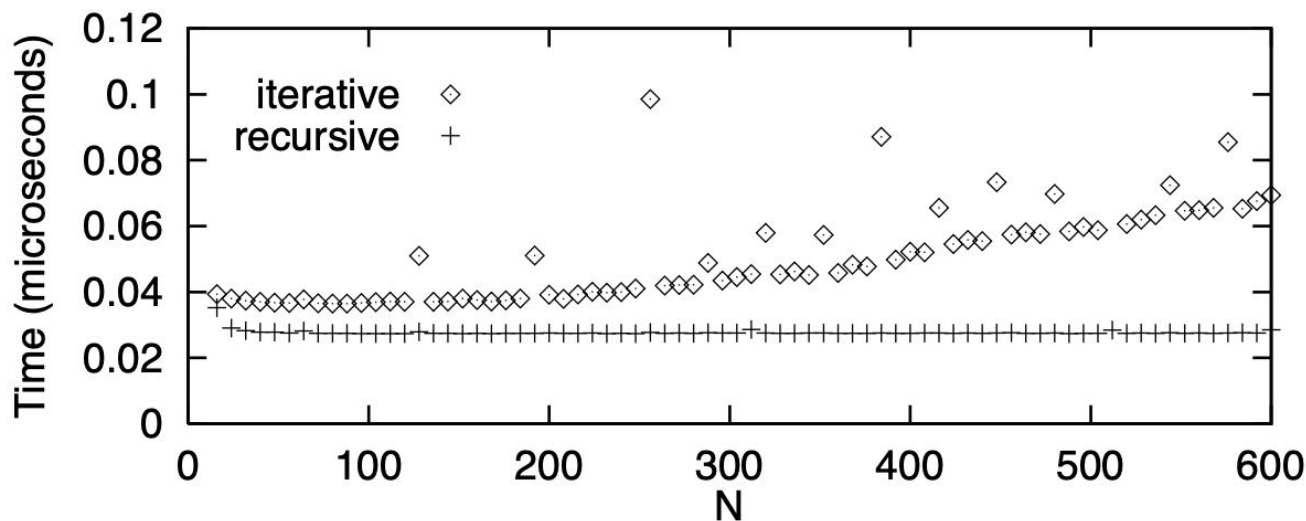


Fig. 5: Average time taken to multiply two $N \times N$ matrices, divided by N^3 .



When to stop recursion?

From the paper:

- When $m=n=p=1$

From a handout by Steven Johnson at MIT:

- When $(m+n+p \leq 48)$
- Probably based on cache sizes of the era
- Is this a cache oblivious approach if we have a stop-recursion condition that is based on cache size?

Key Findings

Cache-oblivious divide-and-conquer achieves multi-level optimal cache complexity (up to constants) for core problems:

- Matrix multiply: $\Theta(N^3 / (B\sqrt{M}))$ misses—matching tuned blocking asymptotically.
- Matrix transpose: $\Theta(N^2 / B)$ misses via recursive partitioning (vs. much worse for naïve).
- Scans, FFT, sorting, and tree layouts attain known optimal bounds without cache parameters.

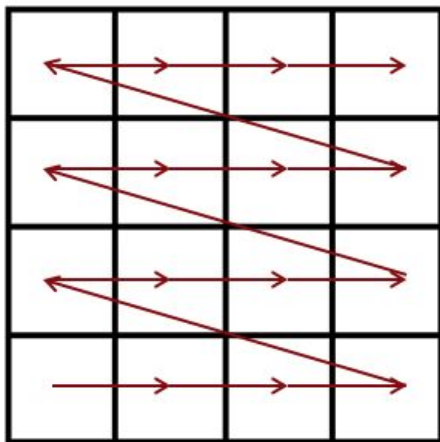
Practical takeaway: good memory layouts emerge from recursion; you often get near-blocked behavior across L1/L2/L3 automatically.

Caveats: constants/recursion overhead and interaction with SIMD/prefetchers can matter; base-case tuning is still important.

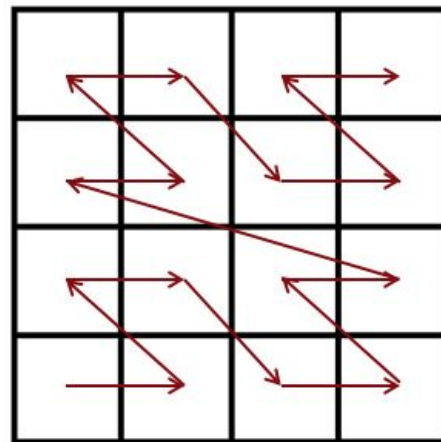
Back to alternate memory layout paper from 2015

Key observation:

- Want: nearby in *index space* also nearby in *physical space*



Row-major order



Z-order

RM index computation

Traditional

$$\text{indx} = i + (j * \text{iSize}) + (k * \text{xSize} * \text{ySize})$$

Zorder-lib

$$\text{indx} = i + \text{joffset}[j] + \text{koffset}[k]$$

2 table lookups, two additions

Table sizes: jSize, kSize

ZO index computation

Traditional

indx = many bit shifts, ANDs, and ORs. How many?

$$\text{ceil}(\log_2(\text{iSize})) + \text{ceil}(\log_2(\text{jSize})) + \text{ceil}(\log_2(\text{kSize}))$$

Zorder-lib

$$\text{indx} = \text{iBits}[i] \mid \text{jBits}[j] \mid \text{kBits}[k]$$

3 table lookups, two ORs

Table sizes:

$$\text{ceil}(\log_2(\text{iSize})), \text{ceil}(\log_2(\text{jSize})), \text{ceil}(\log_2(\text{kSize}))$$

What is Raycasting Volume Rendering?

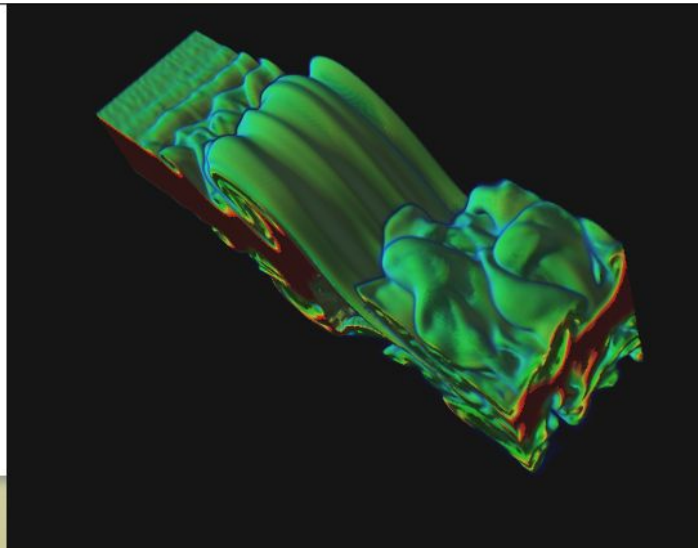
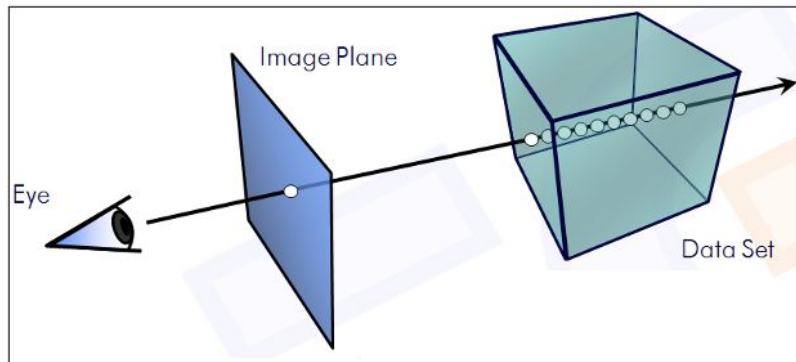
Foreach image pixel

Shoot a ray through the pixel

Where the ray intersects data, compute shade, integrate color, opacity along ray length through the volume

Embarrassingly parallel: all rays may be computed independently

“Semi-structured” memory accesses: each ray accesses data in a predictable way, but each ray’s pattern is different due to path divergence (perspective projection)



Results – Intel IvyBridge (edison)

Runtime:

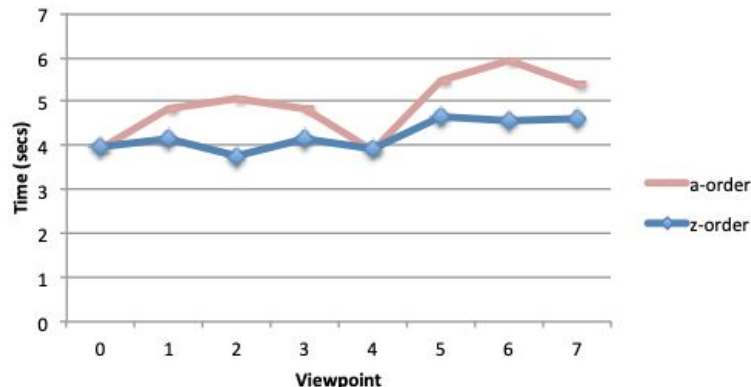
- RM at parity with ZO when rays align well with memory
- ZO faster in all other cases
- ZO performance not affected by viewpoint

Memory utilization:

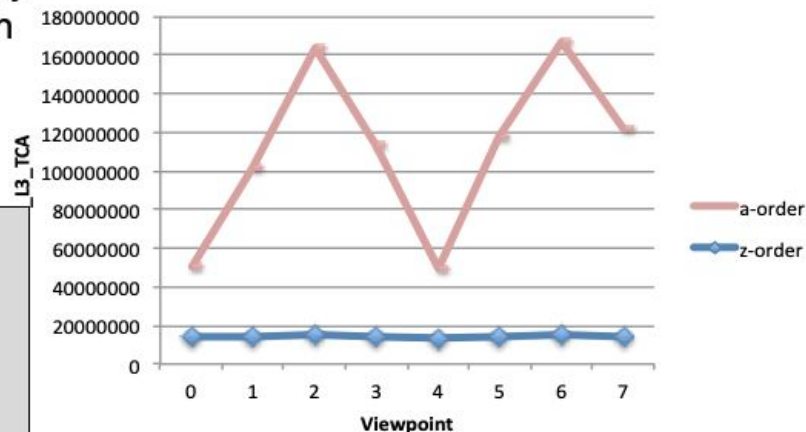
- RM: clear spikes in memory traffic as rays misalign with memory
- PAPI_L3_TCA a good indicator

These results are for a 2-thread case, shown in isolation here to convey the clear correlation between runtime and cache utilization performance.

Volrend, 512^3, IvyBridge, runtime



Volrend, 512^3, IvyBridge, PAPI_L3_TCA



Results – Intel IvyBridge (edison)

Runtime:

- RM at parity with ZO when rays align well with memory
- ZO faster in all other cases, performance not affected by viewpoint

Memory utilization:

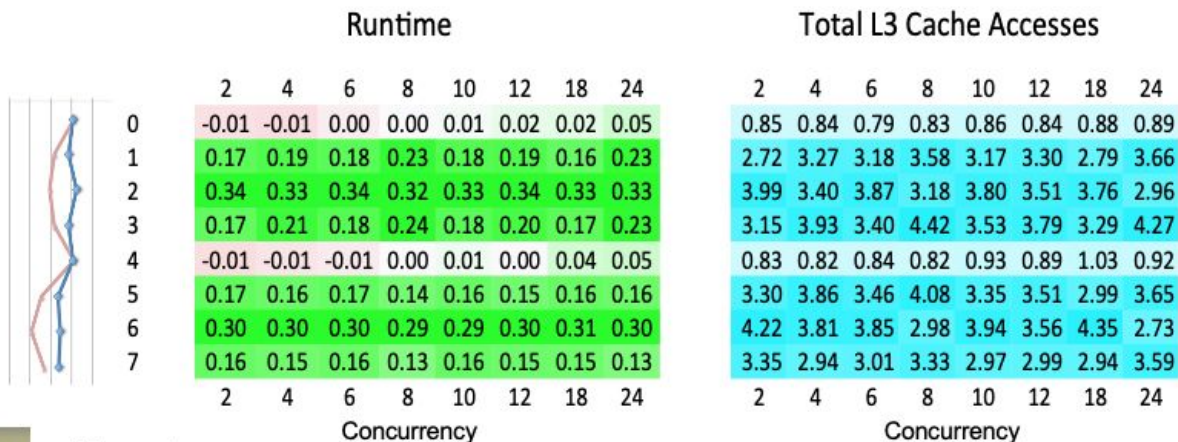
- RM: clear spikes in memory traffic as rays misalign with memory
- ZO: lower L3 TCA in all cases

Scaled differences:

$$d = (a-z)/z$$

>0 : ZO better
<0 : RM better

Volrend, 512^3, IvyBridge: Relative Z- vs. A-Order



Viewpoint

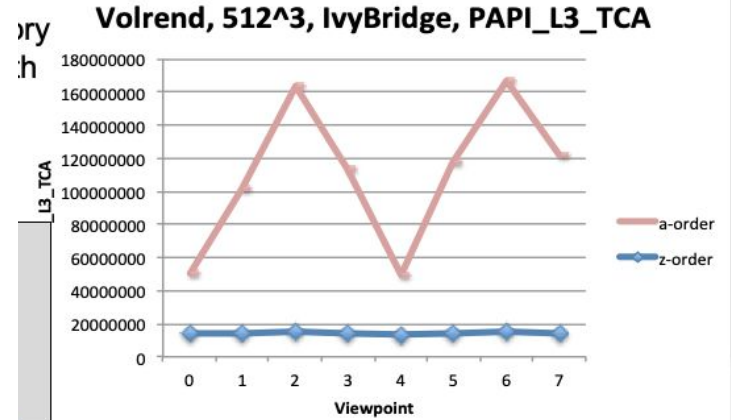
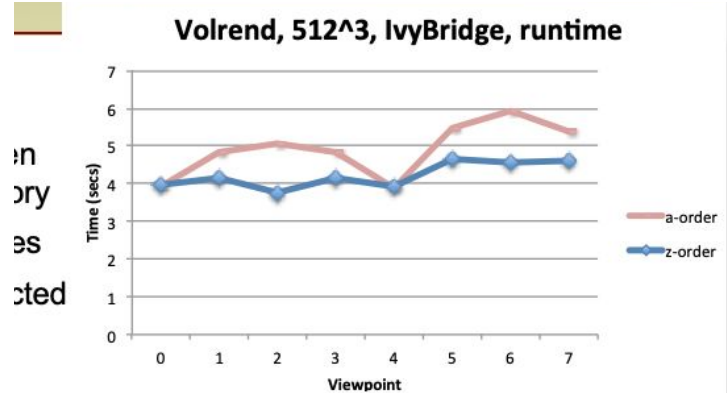
Concurrency: #Threads

Question from Last Time

For viewpoints 5,6,7, why is the runtime higher than for viewpoints 1,2,3 ??

The most likely answer is due to assistance, or not, from the memory prefetcher:

- Runtime is slightly higher
- Memory metrics are also slightly higher
- More pronounced for a-order, but also still present for z-order



Memory Prefetcher

They exist at different levels – L1, L2, LLC

Their job:

- examine/learn memory access patterns,
- predict likely next accesses,
- pull in cache lines ahead of time

Intention:

- Reduce miss rate, lower AMAT

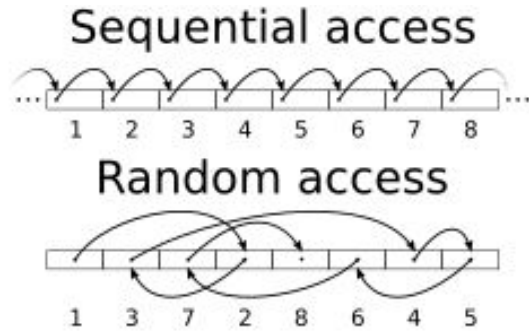
Memory Prefetcher, ctd.

Heuristics:

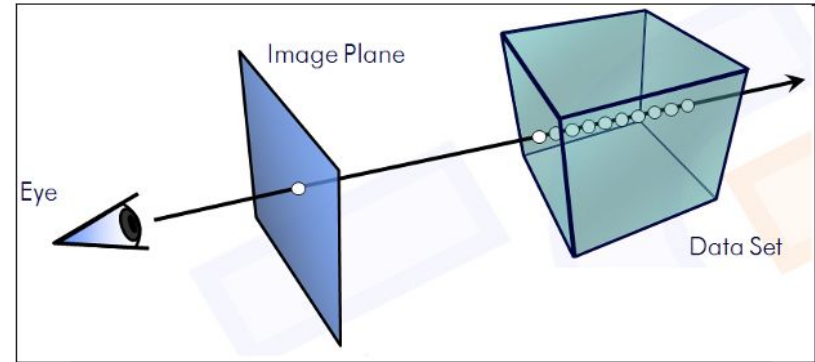
- Next line, adjacent line.
 - On each access (cache line load), prefetch the next line
 - → always forward, good for tight fwd access, useless for backward access
- Stream/sequence detection
 - Detects ascending or descending runs of consecutive line
 - Pulls in next few cache lines
- Stride (IP-based) prefetching:
 - Learns a *constant stride* per instruction
 - Can be positive or negative
- Spatial/region prefetch:
 - When one line of a 128B region arrives in L2, fetch the “pair” line to fill the region
 - AMD/Intel x86 platforms: 64B cache line across L1, L2, L3



Good loops/bad loops



```
for i in 0,N:  
    x += A[i];
```



```
tstart = f(t1);  
tend = f(t2);  
dt = (tend-tstart)/(N-1);
```

```
for i in 0,N:  
    x += A[tstart];  
    tstart += dt;
```

What is Bilateral Filtering?

Edge-preserving smoothing filter

Formulated using:

A gaussian spatial weight: $g(i, \bar{i})$

A gaussian signal weight: $c(i, \bar{i})$

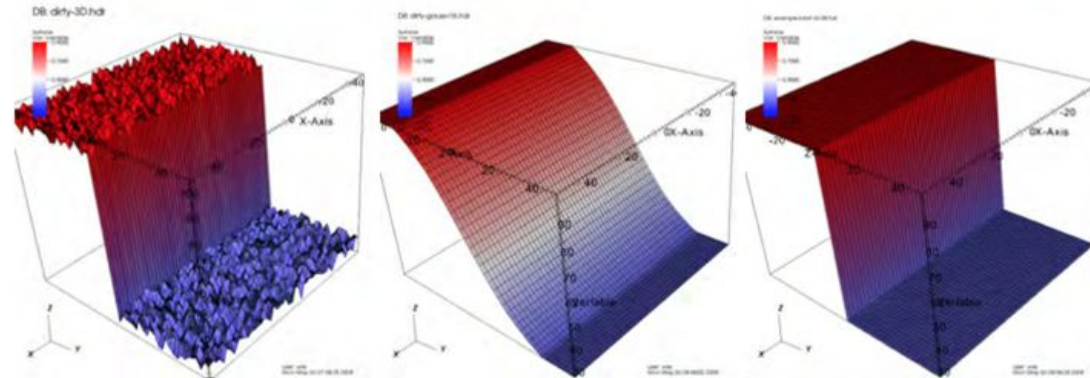
Implementation:

A convolution kernel with some extra computations

$$D(i) = \frac{1}{k(i)} \sum g(i, \bar{i}) c(i, \bar{i})$$

$$g(i, \bar{i}) = e^{-\frac{1}{2} \left(\frac{d(i, \bar{i})}{\sigma} \right)^2}$$

$$k(i) = \frac{1}{\sum g(i, \bar{i}) c(i, \bar{i})}$$



(a) Synthetic, noisy dataset.

(b) Gaussian smoothing ($r = 16$)

(c) Bilateral smoothing ($r = 16$, $\sigma = 15\%$)

Basic idea:

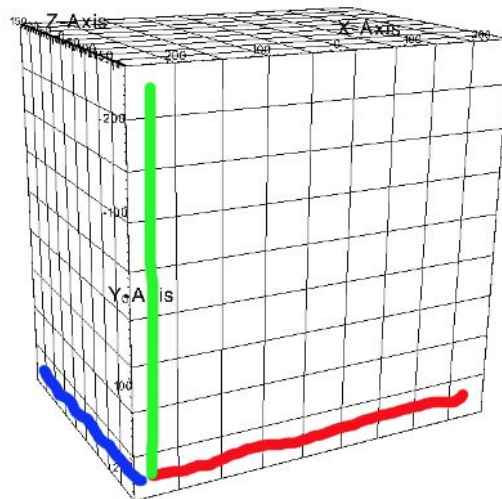
$d[i,j,k] = \text{convolveBilat}(s,i,j,k,<\text{filter parms}>)$

Loop processing order inside the convolution kernel?

- I (x) varies fastest, then J (y), then K (z) – most favorable for RM
- Other options: zyx, yxz, etc.

How parallelized?

- Thread/pencil order: assign to each thread: pencil of voxels, or block of voxels.
- Does width- (px), depth- (pz) height-row assignment matter?
- Not parallelizing the convolution loop, in the OpenMP sense.



Basic idea:

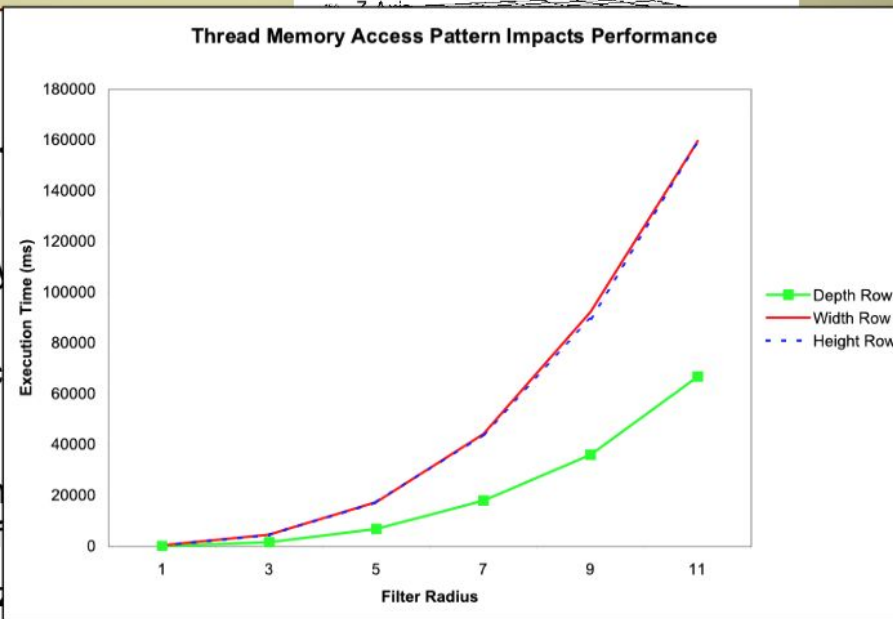
$d[i,j,k] = \text{convolveBilat}(s,i,j,k,$

Processing order inside the convolution:

- I (x) varies fastest, then J (y) most favorable for RM
- Other options: zyx, yxz, etc

How parallelized?

- Thread/pencil order: assign a pencil of voxels, or block of voxels
- Does width- (px), depth- (pz) or height- (py) order



- Thread processing order can make a huge performance difference! This study, run on a GPU (ca 2012) shows that depth-order processing has 2x the performance of either width- or height-order processing. This result is due to better use of the memory hierarchy.

E. W. Bethel, Exploration of Optimization Options for Increasing Performance of a GPU Implementation of a Three-dimensional Bilateral Filter. LBNL TR LBNL-5406E, 2012.

Bilateral3D Results: Intel IvyBridge (edison)

Vary: stencil size, loop processing order, thread/pencil order

Results:

- Except for one configuration (smallest stencil size, processing order most favorable to RM):
 - ZO runtime faster by about 27% to 231%.
 - ZO has significantly fewer L3 TCA, from 0.5 to over 2.0 orders of magnitude.

Scaled differences:

$$d = (a-z)/z$$

>0 : ZO better
<0 : RM better

Bilat3d, 512^3, IvyBridge: Scaled, Relative Difference Z- vs. A-Order

	Runtime							
	2	4	6	8	10	12	18	24
r1 px xyz	-0.02	-0.02	-0.03	-0.03	-0.03	-0.03	-0.04	-0.06
r1 pz zyx	1.62	1.56	1.48	1.43	1.34	1.30	1.21	1.13
r3 px xyz	0.27	0.30	0.42	0.42	0.45	0.45	0.45	0.45
r3 pz zyx	1.04	1.04	1.08	1.08	1.07	1.06	1.22	1.18
r5 px xyz	0.31	0.32	0.33	0.33	0.36	0.36	0.36	0.35
r5 pz zyx	2.23	2.21	2.22	2.24	2.23	2.25	2.29	2.31
	2	4	6	8	10	12	18	24

	Total L3 Cache Accesses							
	2	4	6	8	10	12	18	24
r1 px xyz	-0.87	-0.81	-0.90	-0.88	-0.92	-0.88	-0.91	-0.89
r1 pz zyx	0.63	0.60	0.60	0.54	0.62	0.61	0.77	1.13
r3 px xyz	36.15	26.37	38.26	35.32	43.60	50.78	47.07	26.92
r3 pz zyx	2.84	3.98	3.07	4.32	3.92	3.43	2.24	1.56
r5 px xyz	67.13	69.88	72.03	72.34	75.04	74.80	73.89	69.58
r5 pz zyx	131.43	129.26	127.31	138.44	124.74	133.76	133.34	130.92
	2	4	6	8	10	12	18	24

Concurrency: #Threads

Vary: stencil size, processing order

Results:

- ZO faster than RM in all but one configuration
- ZO shows vastly superior memory utilization characteristics
- L2_READ_DATA_MISS useful as an indicator

Scaled differences:

$$d = (a-z)/z$$

>0 : ZO better

<0 : RM better

Bilat3d, MIC: Scaled, Relative Difference, Z- vs. A-Order

	Runtime				L2_DATA_READ_MISS			
	59	118	177	236	59	118	177	236
r1 px xyz	0.05	0.00	0.03	0.05	-0.28	0.00	0.03	0.05
r1 pz zyx	0.42	0.27	0.23	0.19	8.41	3.03	2.96	5.67
r3 px xyz	0.11	0.31	0.12	0.10	0.72	10.37	23.70	22.86
r3 pz zyx	0.86	0.77	0.84	0.69	2.24	2.76	20.01	12.68
r5 px xyz	1.06	0.58	0.40	0.22	63.40	24.38	39.07	48.50
r5 pz zyx	8.92	6.17	4.93	4.12	207.82	147.62	176.99	618.27
	59	118	177	236	59	118	177	236

Concurrency: #Threads

SFSU MS student Benjamin Kao, CSC 746 Fall 2021

Question: does z-order memory layout offer any performance benefit for matrix-matrix multiply on Intel Knight's Landing and NVIDIA Volta V100 platforms?

Approach:

- Leverage z-order library for MMUL
- Parallelization using OpenMP
- MMUL algorithmic optimization option: block-copy
- Platforms: Intel Knight's Landing, NVIDIA Volta V100 (using OpenMP device offload capabilities)

Kao's Early Z-order MMUL Results: KNL (CPU)

Problem Size (N)	Column Major Runtime (ms)	Z Order Runtime (ms)
128	9.84×10^0	1.18×10^1
256	2.27×10^2	9.17×10^1
512	3.42×10^3	9.15×10^2
1024	2.76×10^4	1.49×10^4
2048	2.21×10^5	1.89×10^5

(left) The Z-order code shows lower runtime in all configurations except N=128 (fits in cache)

Table 1: NERSC Cori KNL Basic MM: Column Major vs Z Order Runtimes.

(Right) As concurrency grows, the difference between ZO and CM codes shrinks.

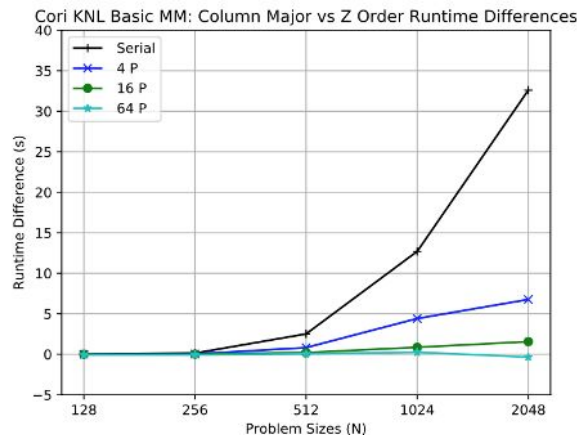


Figure 2: NERSC Cori KNL: Basic Matrix Multiply Runtime Difference (Column Major Runtime - Z order Runtime) between Column Major and Z Order versions at different problem sizes. As problem size increases, the runtime difference increases too.

(Right) CM and ZO show similar levels of performance, with ZO edging out CM in more configurations than not

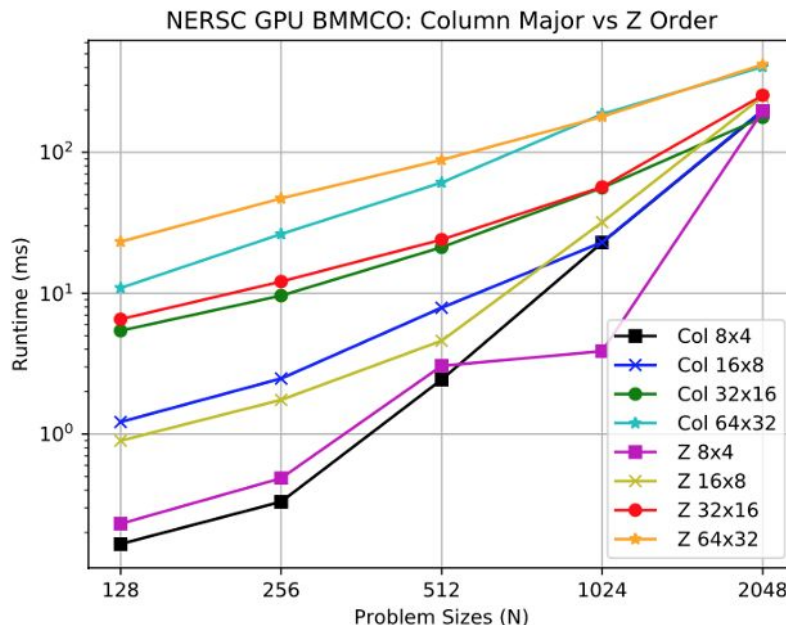


Figure 6: NERSC GPU: BMMCO Runtime of Column Major and Z Order versions. Both version's runtimes are pretty consistent against each other.

An “old idea” still has some useful applications and benefit on modern (then current) platforms (Intel MIC/KNC)

Results show:

- Runtime performance gains
- Better memory system utilization

Limitations:

- Power-of-two size constraints for z-order curve may limit applicability
- Theoretically extends to N dimensions, practical considerations

Important for exascale:

- Reduces data movement in a way that is nearly transparent to the application.
- A “general purpose” idea that can have broad applicability, still.

Interesting historical perspective: D. Wise and J. Frens. Morton-order Matrices Deserve Compilers’ Support. Indiana University TR533, 1999.

Previous works studying cache optimization

The cache performance and optimizations of blocked algorithms. Lam, Rothberg, and Wolf. 4/1991, <https://dl.acm.org/doi/10.1145/106974.106981>

Locality Optimizations for Multi-level caches, Rivera and Tang, 1/1999, <https://dl.acm.org/doi/pdf/10.1145/331532.331534>

Cache-oblivious algorithms. Frigo, Leiserson, Prokop, and Ramchandran. 10/1999. <https://dl.acm.org/doi/10.5555/795665.796479>

What every programmer should know about memory, 11/2007, <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>

What Every Programmer Should Know About Memory (2007)

Key problems:

- What are the practical hardware realities (latency ladders, associativity, prefetchers, TLBs, page sizes, NUMA) that determine whether an access pattern will be fast or slow?
- How should programmers map data layouts and access patterns to these realities?

Key findings

Access regularity rules performance: unit-stride streaming aligns with prefetchers; large strides, pointer chasing, or set collisions defeat them.

TLB behavior matters: strides $\approx$ page size trigger TLB miss storms; huge pages and contiguous layouts mitigate this for streaming workloads.

Writes are special: write-allocate can waste bandwidth for streaming stores; non-temporal stores help for write-only outputs.

Layout choices (AoS $\rightarrow$ SoA), alignment, and padding materially affect vectorization and cache set usage.

NUMA first-touch and CPU affinity can dominate performance at scale; even “serial” phases can suffer from page migration.

A concise practitioner’s checklist: pick a target cache level, enforce unit-stride inner loops, pad to break power-of-two conflicts, measure with counters, and adjust.

The End



A Warner Bros.
PICTURE



SAN FRANCISCO
STATE UNIVERSITY